

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ
ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ КОМП'ЮТЕРНИХ НАУК ТА КІБЕРНЕТИКИ

Звіт до лабораторної роботи №1 з курсу
«Основи Data Mining»

Студента групи ТТП-41
Маркова Максима Юрійовича

викладач: Терещенко Ярослав
Васильович

Київ-2025

Анотація

В цій лабораторній роботі було реалізовано аналіз даних про покупки користувачів на основі частотного аналізу, асоціативних правил та візуалізації результатів.

Основні етапи роботи:

- Попередня обробка даних, включаючи групування транзакцій та перетворення у формат one-hot encoding.
- Дослідження даних, включаючи аналіз частоти появи товарів у кошиках покупців та побудову теплової карти спільних покупок найпопулярніших товарів.
- Побудова графа спільної появи товарів на основі NetworkX.
- Використання алгоритму Apriori для знаходження частих наборів товарів та визначення асоціативних правил між ними.
- Візуалізація результатів, включаючи теплові карти, гістограми та графи асоціативних зв'язків.

Для реалізації було обрано мову програмування Python, оскільки вона містить велику кількість бібліотек для аналізу даних та візуалізації.

Було використано наступні бібліотеки:

- **Pandas** – для зчитування, обробки та аналізу табличних даних.
- **NumPy** – для роботи з числовими масивами та математичними обчисленнями.
- **Matplotlib та Seaborn** – для створення графіків та візуалізації даних.
- **mlxtend** – для реалізації алгоритму Apriori та побудови асоціативних правил.
- **NetworkX** – для побудови та аналізу графів спільної появи товарів.

Отримані результати можуть бути використані для рекомендаційної системи, аналізу поведінки покупців та оптимізації розміщення товарів у магазині.

Вступ

Тема роботи: «Market Basket Analysis».

Мета роботи – дослідити асоціативні зв'язки між товарами, які купують разом, та візуалізувати отримані результати за допомогою сучасних методів аналізу даних.

Лабораторна робота складається з наступних етапів:

1. Попередня обробка даних

- Завантаження та перетворення транзакційного набору даних.
- Використання one-hot кодування для представлення транзакцій у зручному форматі.

2. Дослідження частотних зв'язків між товарами

- Аналіз найбільш популярних товарів у датасеті.
- Побудова матриці спільних покупок.
- Візуалізація даних за допомогою теплових карт та графів зв'язків.

3. Пошук асоціативних правил

- Використання алгоритму Apriori для знаходження частих товарних наборів.
- Визначення асоціативних правил на основі метрик підтримки, довіри та приросту.
- Фільтрація отриманих правил для знаходження найбільш значущих залежностей.

4. Візуалізація отриманих правил

- Побудова графічного представлення правил у вигляді орієнтованого графа.
- Аналіз та інтерпретація знайдених залежностей.
-

1. Теоретична частина та аналіз алгоритму

1.1 Алгоритм Apriori

Apriori — алгоритм глибинного аналізу даних щодо частих одиниць у множинах і машинного навчання щодо асоціативних правил, що застосовується переважно до баз даних транзакцій. Алгоритм ідентифікує елементи/одиниці, що часто повторюються у базі, і розширює їх список до все більших множин з дотриманням правила достатньої частотності. Визначені алгоритмом множини частих одиниць можна використати для визначення правил асоціювання, по яких стають помітними загальні тенденції в базі даних.

В задачі аналізу ринкового кошику одиницями є пропоновані товари, а покупка являє собою транзакцію, яка містить куплені предмети (одиниці). Алгоритм при цьому визначає кореляції такого виду: якщо хтось купує шампунь і лосьйон для гоління, у 90% випадків купується також і піна для гоління.

Дані, що надаються до аналізу, складаються з таблиці транзакцій (на рядках), в якій перераховуються будь-які бінарні одиниці (у колонках). Алгоритм Apriori знаходить співвідношення між множинами одиниць, які зустрічаються у великій частині транзакцій. Правила асоціювання, які отримуються в результаті, мають форму $A \rightarrow B$ при цьому A і B є множинами одиниць, а правило стверджує, що коли у великій частині транзакцій зустрічається множина одиниць A , то там часто зустрічається і множина одиниць B .

1.2 Опис алгоритму Apriori

Алгоритм Apriori було запропоновано Агравалом і Срікантом в 1994 році. Apriori застосовується до баз даних транзакцій (наприклад, наборів товарів, куплених клієнтами, або відвідуваності вебсайту). Кожна транзакція розглядається як множина елементів. Маючи заданий поріг C ,

алгоритм Аргіогі ідентифікує множину елементів, які є підмножинами принаймні S транзакцій в базі даних.

Апріорі використовує підхід «знизу вгору», за якого список частих підмножин розширюється по одному елементу за раз (крок, відомий як генерування кандидатів); відтак групи кандидатів перевіряються на основі наявних даних. Алгоритм завершує роботу, коли подальших успішних розширень знайти неможливо.

Аргіогі використовує пошук у ширину та структуру геш-дерева для ефективного підрахунку елементів-кандидатів множини. Він генерує множини елементів-кандидатів довжиною k з множин довжиною $k-1$. Потім він відсікає кандидатів, які є нечастими підмножинами. Відповідно до леми низхідного змикання (англ. downward closure lemma), множина кандидатів містить усі множини елементів довжини k , які часто зустрічаються. Після цього він сканує базу даних транзакцій, щоб визначити множини елементів, які часто зустрічаються серед кандидатів.

Псевдокод для алгоритму наведено на рис. 1 для бази даних транзакцій T і допоміжного порога ϵ . Застосовується звичайна нотація теорії множин, хоча слід відзначити, що T є мультимножиною. C_k — це множина кандидатів для рівня k . З кожним кроком, як припускається, алгоритм генерує набори кандидатів з великих множин попереднього рівня, дотримуючись леми низхідного закриття. $\text{count}[c]$ звертається до поля структури даних, яка являє собою множину кандидатів c , спочатку вона приймається рівною нулю.

```

Apriori( $T, \epsilon$ )
 $L_1 \leftarrow \{\text{large 1-itemsets}\}$ 
 $k \leftarrow 2$ 
while  $L_{k-1} \neq \emptyset$ 
     $C_k \leftarrow \{a \cup \{b\} \mid a \in L_{k-1} \wedge b \notin a\} - \{c \mid \{s \mid s \subseteq c \wedge |s| = k-1\} \not\subseteq L_{k-1}\}$ 
    for transactions  $t \in T$ 
         $C_t \leftarrow \{c \mid c \in C_k \wedge c \subseteq t\}$ 
        for candidates  $c \in C_t$ 
             $\text{count}[c] \leftarrow \text{count}[c] + 1$ 
     $L_k \leftarrow \{c \mid c \in C_k \wedge \text{count}[c] \geq \epsilon\}$ 
     $k \leftarrow k + 1$ 
return  $\bigcup_k L_k$ 

```

Рисунок 1 - псевдокод для алгоритму Apriori

1.3 Обмеження алгоритму Apriori

Алгоритм Apriori страждає від низької ефективності та компромісів, що призвело до виникнення інших алгоритмів. Утворення кандидата породжує велику кількість підмножин (алгоритм намагається завантажити в набір кандидатів якомога більше даних перед кожним скануванням). Пошук підмножин проходив від низу до верху (по суті обхід в ширину підмножини решітки) знаходить будь-яку максимальну підмножину S тільки після того, як будуть знайдено її $2^{|S|} - 1$ власних підмножин.

Алгоритм сканує базу даних дуже багато разів, що суттєво скорочує швидкодію. Тому, для того щоб алгоритм працював швидко, потрібно, щоб база постійно знаходилась у пам'яті.

Також часова та просторова складність алгоритму є дуже високими.

1.4 Основні метрики алгоритму Apriori

Support

Support елемента x - це не що інше, як відношення числа транзакцій з товаром x до загального числа транзакцій.

Confidence

Confidence ($x \Rightarrow y$) позначають можливість купівлі товару y при купівлі товару x . У цьому методі враховується популярність товару x .

Lift

Lift ($x \Rightarrow y$) — це «цікавість» або ймовірність покупки товару y при купівлі товару x . На відміну від confidence ($x \Rightarrow y$), в цьому методі враховується популярність товару y .

Якщо $\text{lift}(x \Rightarrow y) = 1$, то кореляції в наборі товарів немає.

Якщо $\text{lift}(x \Rightarrow y) > 1$, кореляція в наборі товарів позитивна, тобто ймовірність спільної покупки товарів x і y вище.

Якщо $\text{lift}(x \Rightarrow y) < 1$, кореляція в наборі товарів негативна, тобто сумісна покупка товарів x і y маловірогідна.

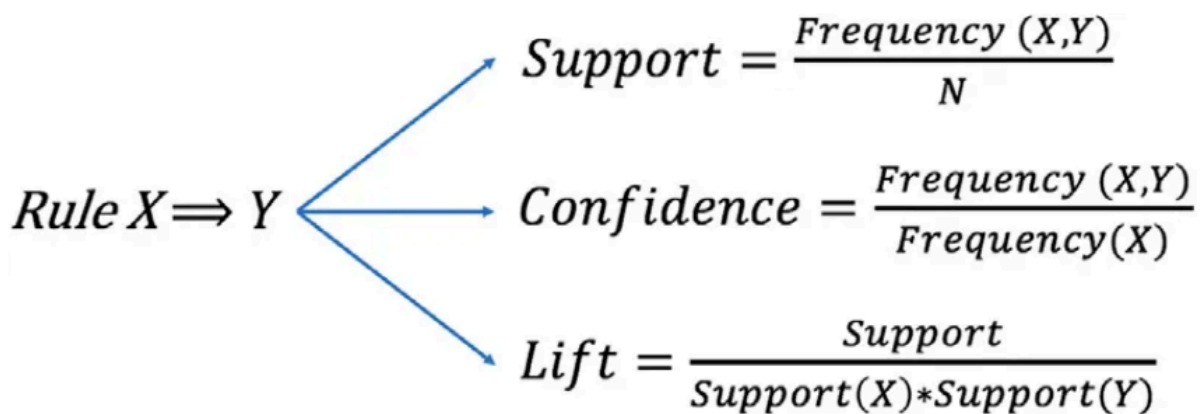


Рисунок 2 - формули Support, Lift та Confidence для асоціативного правила $X \Rightarrow Y$

1.5 One-hot encoding

One Hot Encoding — це метод перетворення категоріальних змінних у двійковий формат. Він створює нові стовпці для кожної категорії, де 1 означає, що категорія присутня, а 0 означає, що її немає. Основна мета One Hot Encoding — забезпечити ефективне використання категоріальних даних у моделях машинного навчання.

					
Car_model			Car_model_BMW	Car_model_Lexus	Car_model_Toyota
0	Toyota		0.0	0.0	1.0
1	Lexus		0.0	1.0	0.0
2	BMW		1.0	0.0	0.0
3	Toyota		0.0	0.0	1.0

Рисунок 3 - приклад One-hot encoding

2. Реалізація

2.1 Data Preprocessing

Датасет поданий у форматі .csv з полями:

1. Member_number - ID of customer
2. Date - Date of purchase
3. itemDescription - Description of product purchased

```
1 Member_number,Date,itemDescription
2 1808,21-07-2015,tropical fruit
3 2552,05-01-2015,whole milk
4 2300,19-09-2015,pip fruit
5 1187,12-12-2015,other vegetables
6 3037,01-02-2015,whole milk
7 4941,14-02-2015,rolls/buns
8 4501,08-05-2015,other vegetables
9 3803,23-12-2015,pot plants
10 2762,20-03-2015,whole milk
11 4119,12-02-2015,tropical fruit
12 1340,24-02-2015,citrus fruit
13 2193,14-04-2015,beef
14 1997,21-07-2015,frankfurter
15 4546,03-09-2015,chicken
16 4736,21-07-2015,butter
17 1959,30-03-2015,fruit/vegetable juice
18 1974,03-05-2015,packaged fruit/vegetables
```

Рисунок 4 - вміст файлу Groceries_dataset.csv

Використовуємо бібліотеку pandas для роботи з датасетом. Групуємо транзакції за Member_number та Date, щоб отримати списки елементів для кожної транзакції. Перетворюємо дані в one-hot encoding для роботи алгоритму Apriori за допомогою об'єкту класу TransactionEncoder() бібліотеки mlxtend. Отримуємо дані, які зображено на рис. 5.

```
Total Transactions: 14963
Instant food products  UHT-milk  ...  yogurt  zwieback
0                      False      False  ...    True    False
1                      False      False  ...   False   False
2                      False      False  ...   False   False
3                      False      False  ...   False   False
4                      False      False  ...   False   False

[5 rows x 167 columns]
```

Рисунок 5 - дані на вхід алгоритму Apriori

2.2 Exploratory Data Analysis (EDA)

Порахувавши загальні кількості кожного продукту на основі one-hot encoding table, відобразимо bar chart найпопулярніших товарів, який зображено на рис. 6.

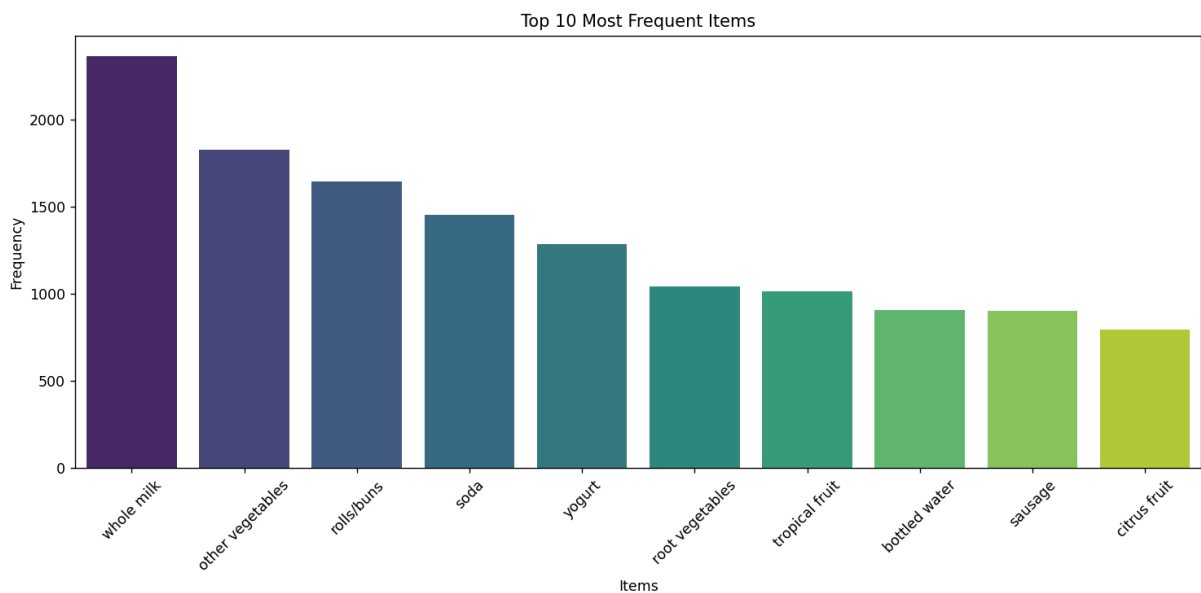


Рисунок 6 - найпопулярніші товари в датасеті

Маючи вміст корзин покупців, обчислимо матрицю спільної появи десяти найпопулярніших товарів та відобразимо її за допомогою бібліотеки seaborn. Вигляд матриці зображено на рис. 7.

Co-occurrence of Top 10 Items										
whole milk	148	130	84	71	57	62	101	80	16	25
other vegetables	130	75	70	95	50	50	22	27	12	17
rolls/buns	84	70	74	79	69	35	22	38	5	10
soda	71	95	79	63	44	54	14	42	6	7
yogurt	57	50	69	44	51	49	14	28	6	12
root vegetables	62	50	35	54	49	32	15	18	9	24
tropical fruit	101	22	22	14	14	15	19	39	13	22
bottled water	80	27	38	42	28	18	39	25	3	8
sausage	16	12	5	6	6	9	13	3	22	14
citrus fruit	25	17	10	7	12	24	22	8	14	17
	whole milk	other vegetables	rolls/buns	soda	yogurt	root vegetables	tropical fruit	bottled water	sausage	citrus fruit

Рисунок 7 - матриця спільної появи товарів

Наостанок побудуємо Network graph на основі n найпоширеніших пар товарів. Розмір та колір вершини залежить від популярності товару. Ребра з'єднують товари, які часто купують разом. Товщина ребра залежить від кількості появи двох товарів в корзинах. Network graph зображено на рис. 8.

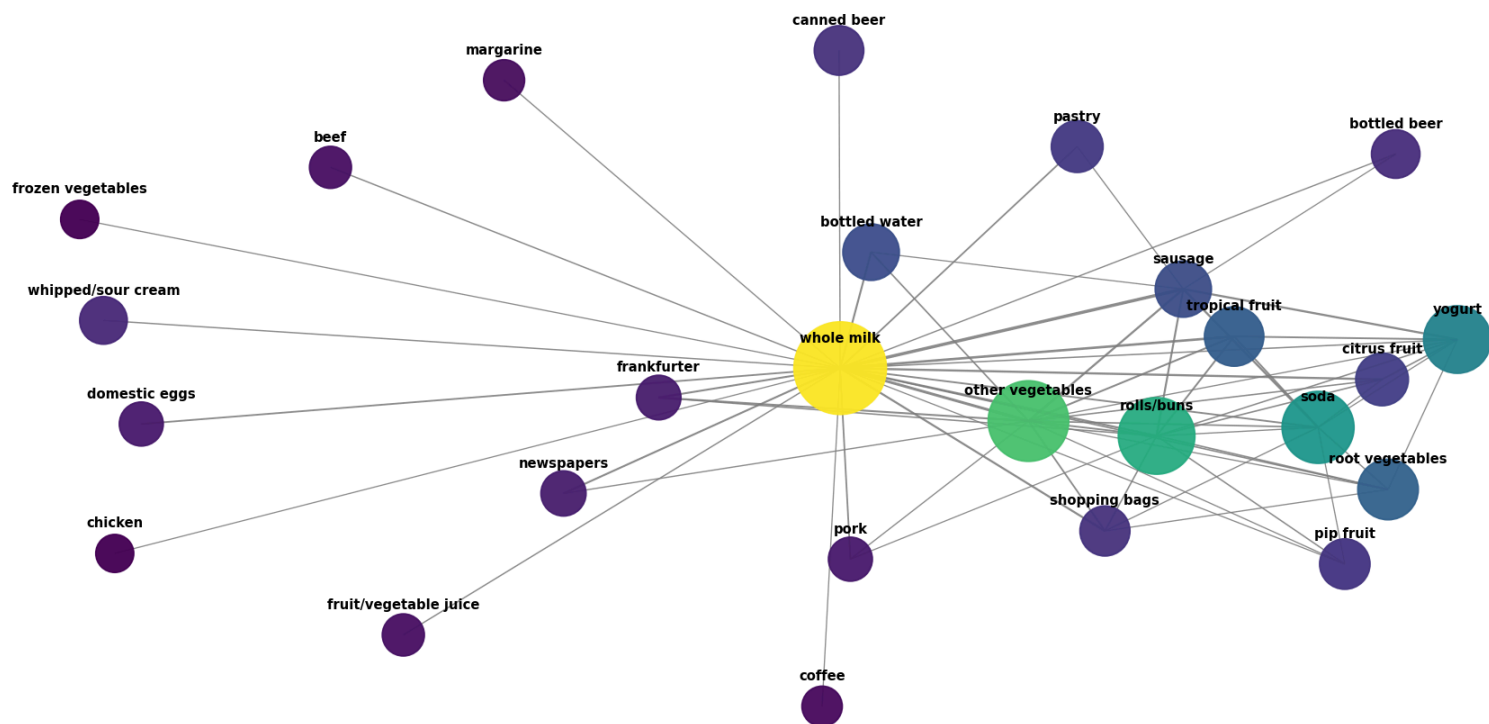


Рисунок 8 - Network graph товарів

2.3 Implementing Apriori Algorithm

Використовуємо бібліотеку mlxtend для імплементції алгоритму Apriori. На вхід подається матриця one-hot encoding з кроку 2.1. Отримуємо найчастіші набори товарів за метрикою support (мінімальний support = 0.001), зображені на рис. 9.

```

Frequent itemsets found: 750
      support      itemsets
0      0.004010      (Instant food products)
1      0.021386      (UHT-milk)
2      0.001470      (abrasive cleaner)
3      0.001938      (artif. sweetener)
4      0.008087      (baking powder)
..      ...
745    0.001136      (rolls/buns, whole milk, sausage)
746    0.001002      (rolls/buns, whole milk, soda)
747    0.001337      (rolls/buns, whole milk, yogurt)
748    0.001069      (sausage, whole milk, soda)
749    0.001470      (sausage, whole milk, yogurt)

```

Рисунок 9 - Найчастіші набори товарів

Далі подаємо цей набір даних на вхід генератора асоціативних правил `association_rules()` бібліотеки `mlxtend`. Отримуємо набір асоціативних правил на основі найчастіших наборів товарів з метриками `support`, `confidence`, `lift`, зображений на рис. 10.

```

Association rules generated: 450
      antecedents      consequents      support      confidence      lift
0      (UHT-milk)      (bottled water)  0.001069      0.050000      0.823954
1      (UHT-milk)      (other vegetables)  0.002139      0.100000      0.818993
2      (UHT-milk)      (rolls/buns)      0.001804      0.084375      0.767013
3      (UHT-milk)      (sausage)      0.001136      0.053125      0.880298
4      (UHT-milk)      (soda)      0.001270      0.059375      0.611444
..      ...
445      (sausage, soda)      (whole milk)  0.001069      0.179775      1.138374
446      (whole milk, soda)      (sausage)  0.001069      0.091954      1.523708
447      (sausage, whole milk)      (yogurt)  0.001470      0.164179      1.911760
448      (sausage, yogurt)      (whole milk)  0.001470      0.255814      1.619866
449      (whole milk, yogurt)      (sausage)  0.001470      0.131737      2.182917

```

Рисунок 10 - Асоціативні правила

2.4. Analysis of Results

Можемо інтерпретувати правила як “if ... then ...”. Наприклад, якщо людина купує сосиски та йогурт, то вона ймовірно купить незбиране молоко з відповідними значеннями метрик.

Не всі правила нам важливі, тому можемо відфільтрувати їх за значеннями метрик `confidence` та `lift`. Отримуємо меншу множину асоціативних правил, яка зображена на рис. 11.

	antecedents	consequents	...	confidence	lift
13	(berries)	(other vegetables)	...	0.122699	1.004899
24	(beverages)	(soda)	...	0.112903	1.162678
100	(chewing gum)	(yogurt)	...	0.116667	1.358508
109	(chocolate)	(rolls/buns)	...	0.118980	1.081592
155	(detergent)	(rolls/buns)	...	0.116279	1.057037
156	(detergent)	(whole milk)	...	0.162791	1.030824
157	(detergent)	(yogurt)	...	0.124031	1.444261
165	(flour)	(tropical fruit)	...	0.109589	1.617141
167	(frankfurter)	(other vegetables)	...	0.136283	1.116150
174	(frozen meals)	(other vegetables)	...	0.127490	1.044134
188	(fruit/vegetable juice)	(rolls/buns)	...	0.110020	1.000136
205	(ham)	(whole milk)	...	0.160156	1.014142
216	(hard cheese)	(rolls/buns)	...	0.113636	1.033014
221	(herbs)	(yogurt)	...	0.107595	1.252874
228	(ice cream)	(rolls/buns)	...	0.114537	1.041205
245	(meat)	(other vegetables)	...	0.126984	1.039991
272	(oil)	(soda)	...	0.121076	1.246844
286	(pot plants)	(other vegetables)	...	0.128205	1.049991
311	(packaged fruit/vegetables)	(rolls/buns)	...	0.141732	1.288421
332	(processed cheese)	(rolls/buns)	...	0.144737	1.315734
333	(processed cheese)	(root vegetables)	...	0.105263	1.513019
335	(red/blush wine)	(rolls/buns)	...	0.127389	1.158028
340	(seasonal products)	(rolls/buns)	...	0.141509	1.286395
383	(semi-finished bread)	(whole milk)	...	0.176056	1.114825
402	(soft cheese)	(yogurt)	...	0.126667	1.474952
423	(other vegetables, soda)	(rolls/buns)	...	0.117241	1.065785
424	(other vegetables, rolls/buns)	(soda)	...	0.107595	1.108013
425	(soda, rolls/buns)	(other vegetables)	...	0.140496	1.150651
435	(whole milk, sausage)	(rolls/buns)	...	0.126866	1.153275
437	(sausage, rolls/buns)	(whole milk)	...	0.212500	1.345594
441	(whole milk, yogurt)	(rolls/buns)	...	0.119760	1.088685
443	(yogurt, rolls/buns)	(whole milk)	...	0.170940	1.082428
444	(whole milk, sausage)	(soda)	...	0.119403	1.229612
446	(sausage, soda)	(whole milk)	...	0.179775	1.138374
447	(whole milk, yogurt)	(sausage)	...	0.131737	2.182917
448	(sausage, yogurt)	(whole milk)	...	0.255814	1.619866
449	(whole milk, sausage)	(yogurt)	...	0.164179	1.911760

Рисунок 11 - Відфільтрована множина правил

2.5 Visualization

Для візуалізації результатів можемо обрати декілька інструментів:

- **Діаграми розсіювання**

Покажемо залежність Confidence від Support. Додатковим показником буде Lift, значення якого буде відображатись через розмір та колір точки на графіку. Діаграма розсіювання зображена на рис. 12. Залежність Lift від Support зображена на рис. 13.

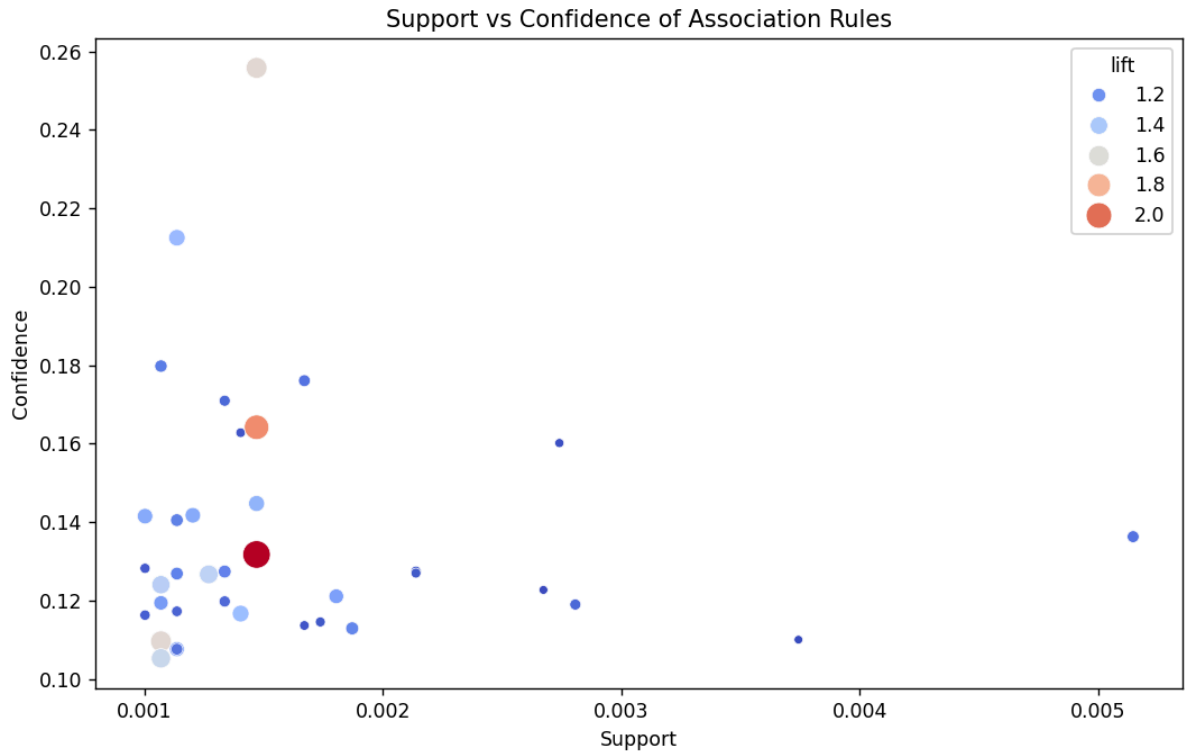


Рисунок 12 - Діаграма розсіювання *Support vs Confidence*

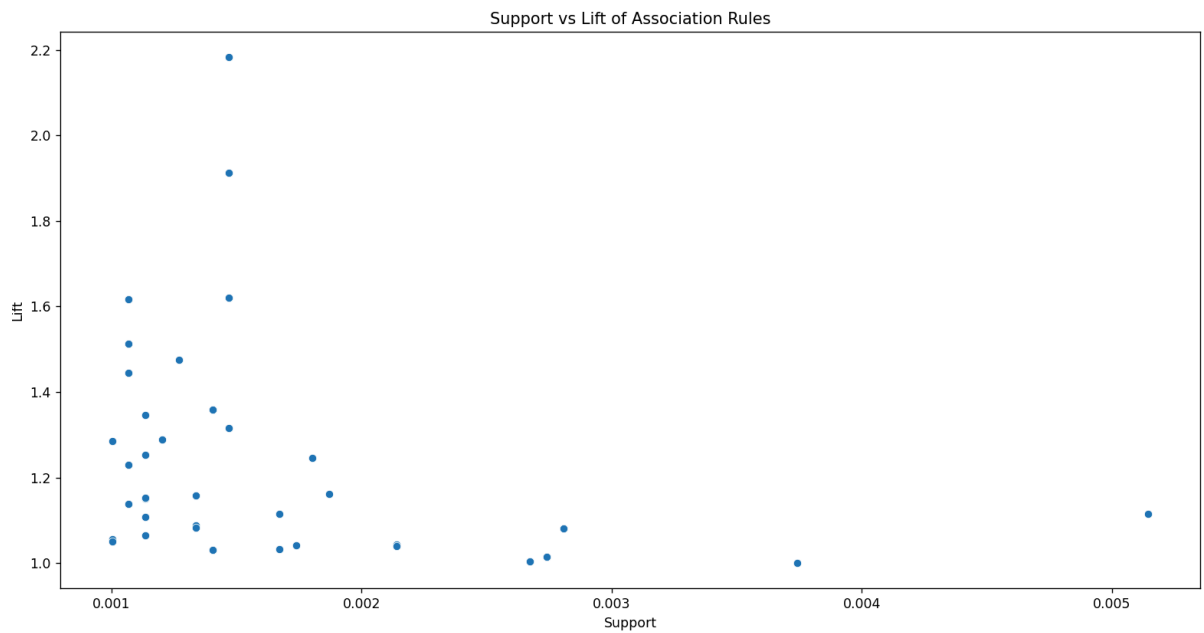


Рисунок 13 - Залежність *Lift* від *Support*

- **Орієнтований мережевий граф**

Для відображення зв'язків між товарами на основі відфільтрованих асоціативних правил використаємо орієнтований мережевий граф. Орієнтоване ребро графа буде показувати зв'язок

“якщо ..., то ...”. Вершинами графа будуть товари. Колір та розмір вершини будуть залежати від її степеню. Товщина ребра буде залежати від метрики Confidence. Отриманий орієнтований мережевий граф зображено на рис. 14.

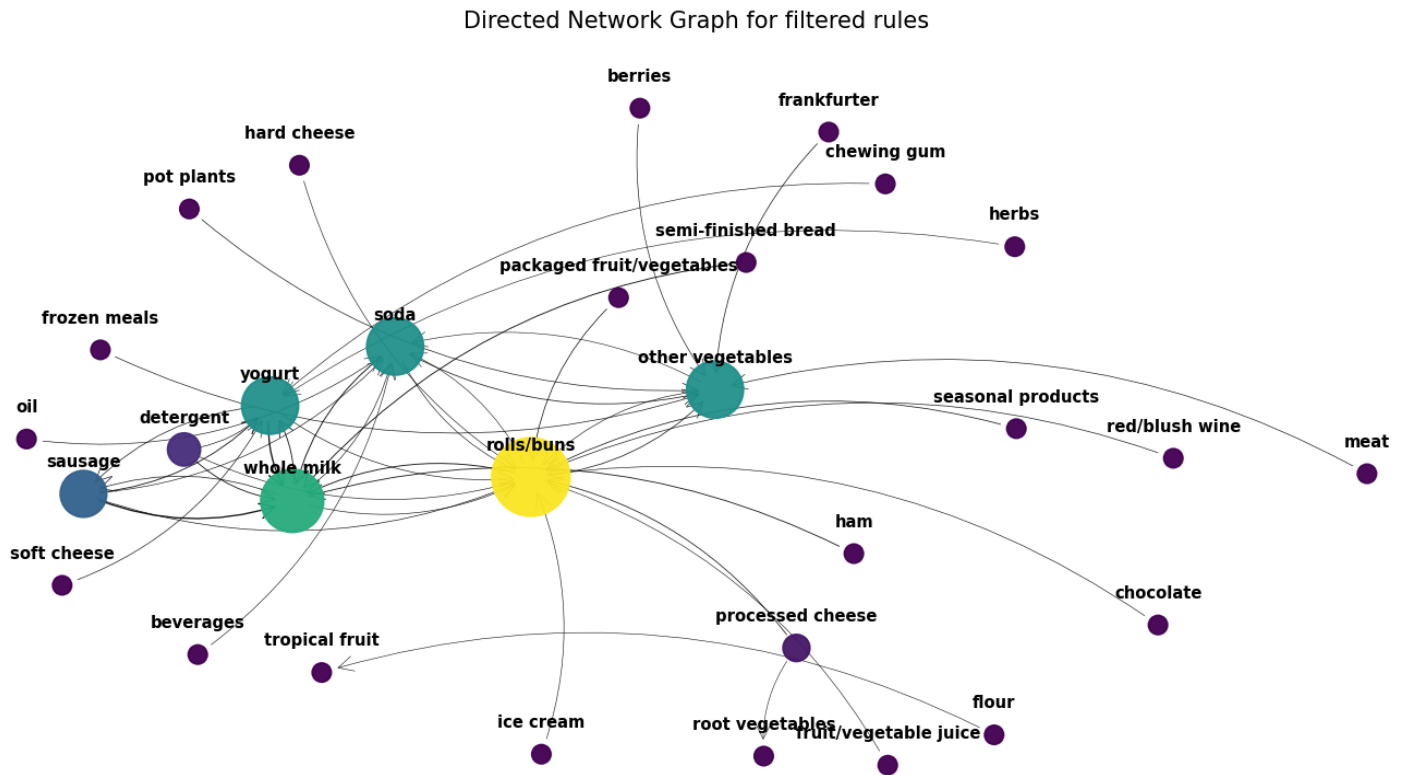


Рисунок 14 - Орієнтований мережевий граф

Зауважимо, що зміна параметрів фільтрування впливає на результати візуалізації, що наведено на рисунках нижче.

1. Confidence > 0.1 & Lift > 1.5

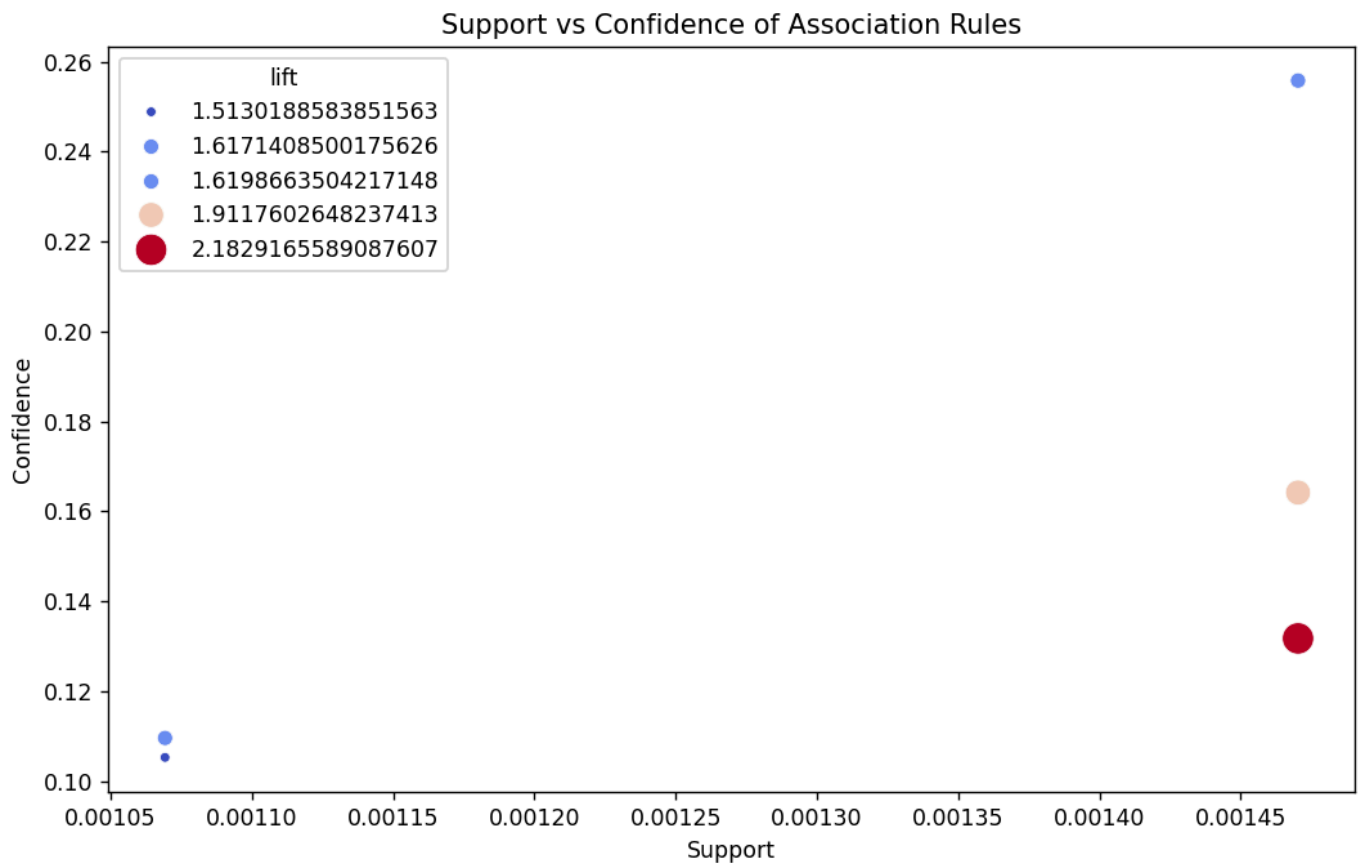


Рисунок 15 - Діаграма розсіювання *Support vs Confidence*

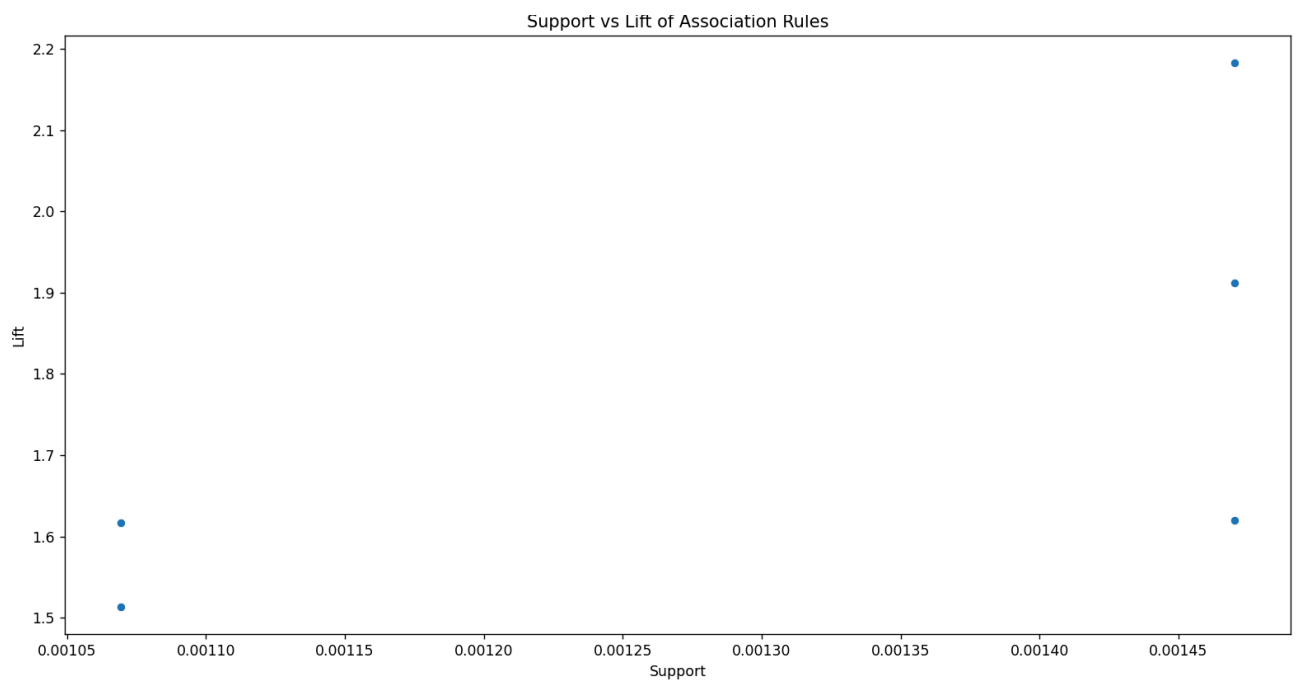


Рисунок 16 - Залежність *Lift* від *Support*

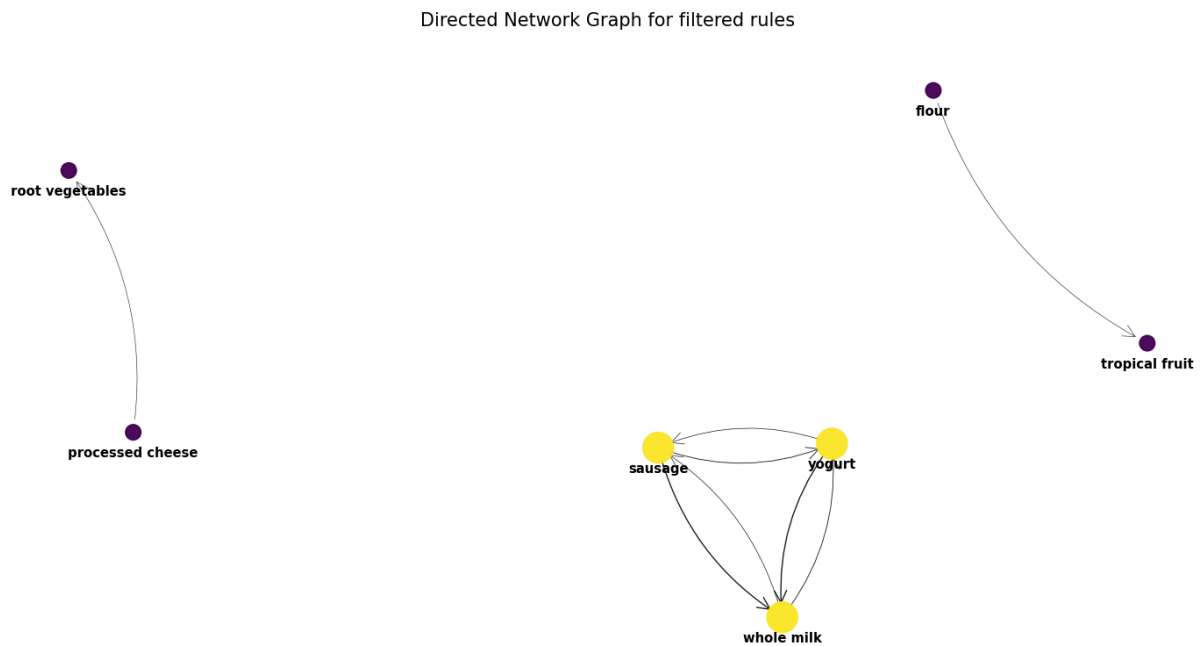


Рисунок 17 - Орієнтований мережевий граф

Бачимо, що для наборів асоціативних правил з позитивною кореляцією товарів можна виділити основні, які зображені на рис. 18. Наприклад, якщо людина купляє незбиране молоко та сосиски, то вона скоріш за все купить йогурт.

	antecedents	consequents	support	confidence	lift
165	(flour)	(tropical fruit)	0.001069	0.109589	1.617141
333	(processed cheese)	(root vegetables)	0.001069	0.105263	1.513019
447	(yogurt, whole milk)	(sausage)	0.001470	0.131737	2.182917
448	(sausage, whole milk)	(yogurt)	0.001470	0.164179	1.911760
449	(yogurt, sausage)	(whole milk)	0.001470	0.255814	1.619866

Рисунок 18 - Відфільтровані асоціативні правила

2. Confidence > 0.04 & Lift > 1

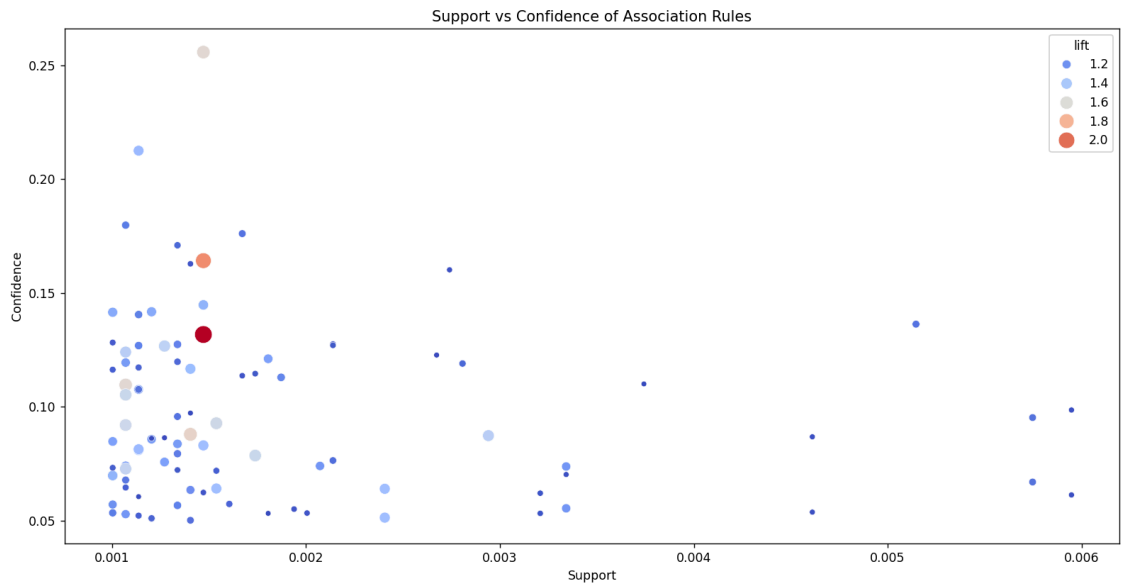


Рисунок 19 - Діаграма розсіювання Support vs Confidence

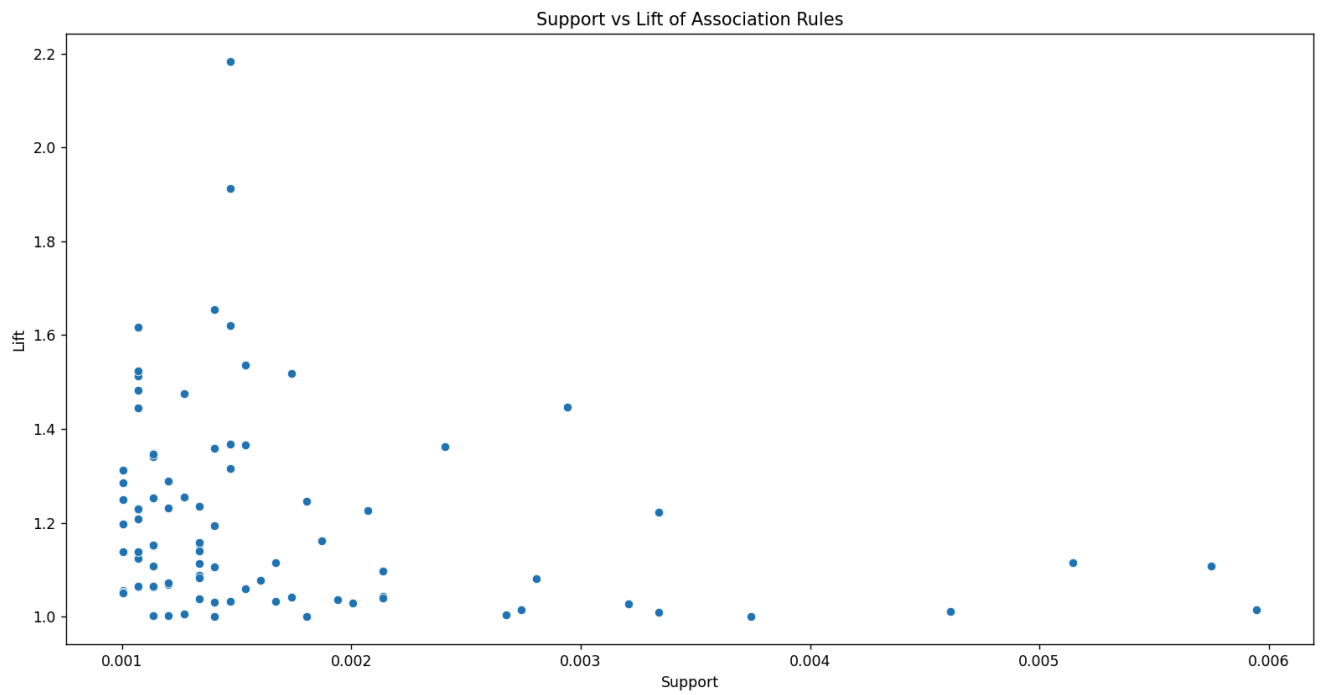


Рисунок 20 - Залежність Lift від Support

[illegible]

Рисунок 21 - Орієнтований мережевий граф

Висновки

В результаті виконання лабораторної роботи було розроблено функціональний додаток, що здійснює аналіз частих множин та асоціативних правил у наборі транзакційних даних. Додаток дозволяє визначати популярні товари, будувати графи зв'язків між ними та формувати асоціативні правила, які можуть бути використані для аналізу споживчих уподобань.

Перспективою розвитку даного застосунку є розширення функціональності шляхом підтримки альтернативних алгоритмів добування асоціативних правил, таких як FP-Growth, та інтерактивна візуалізація результатів. Додатково, можна інтегрувати можливість вибору параметрів підтримки та довіри для гнучкішого налаштування аналізу.

Для аналізу транзакційних даних було використано алгоритм Apriori, а також виконано такі завдання:

- Проведено аналіз даних та виконано попередню обробку транзакцій.
- Побудовано візуалізації найпопулярніших товарів та їх співзустрічованості.
- Реалізовано алгоритм Apriori для знаходження частих множин товарів.
- Згенеровано асоціативні правила та виконано їх фільтрацію за метриками support, confidence та lift.
- Створено графи залежностей товарів та візуалізовано отримані результати.

Розроблений додаток може бути використаний для аналізу споживчих трендів, розробки рекомендаційних систем та оптимізації маркетингових стратегій.

Додатки

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import numpy as np
from itertools import combinations
from collections import Counter

# Load the dataset
df = pd.read_csv("Groceries_dataset.csv")

# 1. Data Preprocessing

# Group purchases by customer and date to obtain a list of items per transaction
transactions = df.groupby(["Member_number",
"Date"])[["itemDescription"]].apply(list).tolist()

# Use TransactionEncoder to transform the transaction list into a one-hot
encoded format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

#print(df.head())
#print(transactions)
#print(te.columns_[:5])
#print(f'Total Transactions: {len(transactions)}')
#print(df_encoded.head())

# 2. Exploratory Data Analysis (EDA)

# Count the occurrence of each item
item_counts = df_encoded.sum().sort_values(ascending=False)
```

```

# Plot the top 10 most popular items
plt.figure(figsize=(12, 6))
sns.barplot(x=item_counts[:10].index, y=item_counts[:10].values,
palette="viridis")
plt.xticks(rotation=45)
plt.title("Top 10 Most Popular Items")
plt.xlabel("Items")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Build a co-occurrence matrix for frequently bought-together items
basket_data = df.groupby(["Member_number",
"Date"])[["itemDescription"]].apply(list)

# Generate pairs of items that appear together in a transaction
item_pairs = []
for items in basket_data:
    item_pairs.extend(combinations(items, 2))

# Count the frequency of item pairs
pair_counts = Counter(item_pairs)
item_counts = Counter(df["itemDescription"])

# Select the top 10 most popular items
n = 10
top_n_items = [item for item, _ in item_counts.most_common(n)]

# Create a co-occurrence matrix for the top 10 items
co_occurrence_matrix = pd.DataFrame(index=top_n_items,
columns=top_n_items).fillna(0)
for (item1, item2), count in pair_counts.items():
    if item1 in top_n_items and item2 in top_n_items:
        co_occurrence_matrix.loc[item1, item2] = count
        co_occurrence_matrix.loc[item2, item1] = count
co_occurrence_matrix = co_occurrence_matrix.astype("int64")

```



```

# Display a heatmap of co-occurrence between top 10 items
plt.figure(figsize=(10, 10))
sns.heatmap(co_occurrence_matrix, annot=True, fmt="d", linewidths=0.5,
cbar=False, cmap="coolwarm", square=True)
plt.title("Co-occurrence of Top 10 Items")
plt.tight_layout()
plt.show()

# Build an association graph using NetworkX
top_pairs = dict(pair_counts.most_common(80))
G = nx.Graph()

# Add nodes with their frequency
items_set = {item: item_counts[item] for item1, item2 in top_pairs.keys() for
item in (item1, item2)}
for node, freq in items_set.items():
    G.add_node(node, frequency=freq)

# Add edges between items
for (item1, item2), weight in top_pairs.items():
    if item1 != item2:
        G.add_edge(item1, item2, weight=weight)

# Graph visualization
plt.figure(figsize=(8, 8))
pos = nx.spring_layout(G)
node_size = [items_set[node] * 1.4 for node in G.nodes()]
node_colors = [items_set[node] for node in G.nodes()]
nx.draw_networkx_nodes(G, pos, node_color=node_colors,
cmap=plt.cm.viridis, node_size=node_size, alpha=0.95)
edges = [(u, v) for u, v in G.edges()]
weights = [G[u][v]['weight'] for u, v in edges]
nx.draw_networkx_edges(G, pos, edgelist=edges, width=[w / max(weights) * 2
for w in weights], edge_color="gray", alpha=0.9)
label_pos = {node: (x, y + 0.09) for node, (x, y) in pos.items()}
nx.draw_networkx_labels(G, label_pos, font_size=8.5, font_weight="bold",
font_color="black")

```

```
plt.title("Item Association Graph")
plt.tight_layout()
plt.show()
```

3. Apriori Algorithm for Association Rule Mining

```
# Find frequent itemsets using the Apriori algorithm
frequent_itemsets = apriori(df_encoded, min_support=0.001,
use_colnames=True)
print(f'Found {len(frequent_itemsets)} frequent itemsets')
print(frequent_itemsets)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="confidence",
min_threshold=0.05)
print(f'Generated {len(rules)} association rules')
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])

# Filter rules based on confidence and lift
filtered_rules = rules[(rules['confidence'] > 0.04) & (rules['lift'] > 1)]
print(filtered_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

4. Visualization of Association Rules

```
# Scatter plot: Support vs Confidence
plt.figure(figsize=(10, 6))
sns.scatterplot(x=filtered_rules['support'], y=filtered_rules['confidence'],
size=filtered_rules['lift'], hue=filtered_rules['lift'], palette='coolwarm',
sizes=(20, 200))
plt.title("Support vs Confidence in Association Rules")
plt.xlabel("Support")
plt.ylabel("Confidence")
plt.show()

# Scatter plot: Support vs Lift
sns.scatterplot(filtered_rules, x="support", y="lift")
plt.title("Support vs Lift in Association Rules")
```

```

plt.xlabel("Support")
plt.ylabel("Lift")
plt.show()

# Build a directed graph for association rules
G = nx.DiGraph()
for _, row in filtered_rules.iterrows():
    antecedents = list(row['antecedents'])
    consequents = list(row['consequents'])
    for antecedent in antecedents:
        for consequent in consequents:
            G.add_edge(antecedent, consequent, weight=row['confidence'])

# Graph visualization settings
degree_dict = dict(G.degree())
node_size = [degree_dict[node] * 100 for node in G.nodes()]
node_colors = [degree_dict[node] for node in G.nodes()]
plt.figure(figsize=(16, 16))
pos = nx.spring_layout(G, seed=41, k=0.55)
nx.draw_networkx_nodes(G, pos, node_color=node_colors,
cmap=plt.cm.viridis, node_size=node_size, alpha=0.95)
edges = [(u, v) for u, v in G.edges()]
weights = [G[u][v]['weight'] for u, v in edges]
nx.draw_networkx_edges(
    G, pos, edgelist=edges,
    width=[w * 3 for w in weights],
    edge_color="black", alpha=0.9,
    arrowstyle='->', arrowsize=20,
    connectionstyle="arc3,rad=0.2"
)
label_pos = {node: (x, y + 0.07) for node, (x, y) in pos.items()}
nx.draw_networkx_labels(G, label_pos, font_size=6.5, font_weight="bold",
font_color="black")
plt.title(f"Directed Network Graph for filtered rules")
plt.axis("off")
plt.show()

```

Джерела

1. Алгоритм Apriori [Електронний ресурс] - Режим доступу до ресурсу:
<https://medium.com/nuances-of-programming/%D0%BF%D1%80%D0%BE%D1%81%D1%82%D0%BE%D0%B9-%D1%81%D0%BF%D0%BE%D1%81%D0%BE%D0%B1-%D1%80%D0%B5%D1%88%D0%B8%D1%82%D1%8C-%D0%B0%D0%BB%D0%B3%D0%BE%D1%80%D0%B8%D1%82%D0%BC-apriori-%D1%81-%D0%BD%D1%83%D0%BB%D1%8F-302ec6e7688c>
2. Han, Jiawei; Kamber, Micheline; Pei, Jian (2012). Mining Frequent Patterns, Associations, and Correlations: Basic Concepts and Methods, p 243-278.
3. Hahsler, Michael (2005). "Introduction to arules – A computational environment for mining association rules and frequent item sets"
[Електронний ресурс] - Режим доступу до ресурсу:
<https://web.archive.org/web/20190430193743/https://mran.revolutionanalytics.com/web/packages/arules/vignettes/arules.pdf>
4. Wong, Pak (1999). "Visualizing Association Rules for Text Mining"
[Електронний ресурс] - Режим доступу до ресурсу:
<https://neuro.bstu.by/ai/Data-mining/Stock-market/InfoVis1999Association.pdf>
5. Mlxtend [Електронний ресурс] - Режим доступу до ресурсу:
<https://rasbt.github.io/mlxtend/>